COSC 6385 Assignment 1

Ali Raza

Code repository: `https://github.com/ali-raza-0076/CompArch-Assignment-One`

Part 1 (`branch-prediction/`) implements a Pentium M-style branch direction predictor in the CBP2 infrastructure, plus a complete predictor with target prediction for extra credit. Part 2 (`o3-simulator/`) extends the provided Python Tomasulo simulator to issue 1 to 4 instructions per cycle in order, with branch prediction and full misprediction recovery.

Part 1: Pentium M-Style Hybrid Branch Direction Predictor

1 Design

The required predictor (`pm_predictor`) combines two components. The bimodal table is a direct-mapped array of 4096 two-bit saturating counters, indexed by the low 12 bits of the branch address, with no tag. The global table adds correlation: it folds in a global history register (GHR), a shift register of recently resolved outcomes regardless of which static branch produced them, using a gshare-style hash:

$$h = \text{addr}[14{:}0] \oplus \text{GHR}[14{:}0], \quad \text{index} = h[14{:}6], \quad \text{tag} = h[5{:}0] \tag{1}$$

This gives a 9-bit index (512 sets) and 6-bit tag, matching the widths in the assignment's diagram. The table is 4-way, with true LRU replacement (a small per-set recency ordering). On a tag hit, the global counter supplies the prediction; otherwise the bimodal table does, the same precedence the real hardware uses. Both tables train on every resolved conditional branch. Unconditional branches are always predicted taken and train nothing, matching the reference `gshare` predictor already in the codebase.

I started with a 2-way global table, since the assignment explicitly allows that simplification, and only later went back and tested the full 4-way design to see whether it was worth the extra fidelity. It was: average direction MPKI across all 19 traces dropped from 7.254 to 6.604 (9%), just from halving how often unrelated branches evict each other's entries. A second idea, folding a longer raw history register into the same 15-bit hash (a standard technique for predictors wanting more history than their table width can index directly), made things *worse* (7.174 MPKI with a 22-bit history), since the extra history added aliasing without adding table capacity, and was reverted. Both results are reported because the second one is still informative, even though it isn't part of the final predictor.

Getting the infrastructure itself to run took two fixes unrelated to the predictor: the trace reader referenced decompressor binaries by hardcoded Unix paths that don't exist on Windows, and once that was fixed, it still silently truncated every trace a few bytes in, because the decompressed pipe

defaults to text mode on this platform and a byte that occurs naturally in the compressed stream gets read as end-of-file. Both are fixed at the point the trace file opens, conditionally compiled so a POSIX build is unaffected.

2 Verification

Before trusting any aggregate number, I worked through a short sequence by hand: a branch at address $0x44$, always taken, across its first three occurrences from a fresh predictor, compared against the compiled predictor's actual behavior (Table 1).

Table 1: A branch at $0x44$, repeated three times, traced by hand.

Step	Hash	Tag	Index	Prediction	GHR after
1	68⊕0=68	4	1	not taken (global miss, bimodal fallback)	1
2	68⊕1=69	5	1	not taken (tag 4≠5, still a miss)	3
3	68⊕3=71	7	1	**taken** (bimodal counter reaches 2)	7

The index stays fixed since a GHR under 64 only perturbs the hash's low 6 bits; the tag changes every time. By step 3, both ways of the relevant set are occupied by tags 4 and 5, neither matching tag 7, correctly still a miss. It's the bimodal counter, now at its second increment, that flips the prediction. I'd first written down the wrong expected answer for this step (forgetting the bimodal counter had already climbed to 2), and catching that before comparing against the real output was the point of doing this by hand. Corrected, it matched exactly, as did a 100,000-iteration run checking no counter ever escapes its 0–3 range.

Table 2 gives the full result across all 19 traces, compared trace by trace against the reference `gshare` predictor (32K entries, untagged, unmodified code already in the infrastructure). The two trade wins roughly evenly, with the hybrid predictor using well under a third of gshare's counter storage ($4096 + 512\times4 = 6144$ counters against 32,768).

3 Extra Credit: Target Prediction

I extended the predictor with target prediction: a 512-set, 4-way BTB for direct branches (indexed by address bits [12:4], tagged by bits [21:13], with a 4-bit offset), and a smaller 256-set, 4-way indirect target cache hashed by address and GHR. The direction side delegates to the required predictor via composition, rather than duplicating its logic, so the two parts can't drift apart.

The first working version compiled and ran, but target MPKI came out near 150 on several traces, despite the BTB hitting over 99% of lookups when I instrumented it directly. Two separate problems turned out to be responsible. First, I had assumed the trace format's "target" field meant a branch's fixed jump destination; a quick diagnostic logging repeated occurrences of the same branch showed it actually alternates between the jump target (taken) and the fall-through address (not taken). My BTB only ever learned the taken-side value, so every not-taken occurrence scored against a value it had no way to predict. Fixed by storing both per entry. Second, that fix alone didn't move the numbers. The actual predicted values were correct by direct inspection, yet every harness comparison still counted as a miss. The getter for a predicted target in the given `predictor.h` is declared to return `bool`, while its own setter and backing field are `unsigned int`; every call silently collapsed a real address to 0 or 1 before comparison. This is a bug in the provided infrastructure, not my code: target prediction could never have scored correctly for anyone using it as given. One-line fix, no effect on direction scoring.

Table 2: Direction MPKI, required predictor vs. reference gshare, all 19 traces.

Trace	PM hybrid	gshare 32K
164.gzip	12.461	12.473
175.vpr	14.071	13.415
176.gcc	13.302	11.254
181.mcf	16.490	15.837
186.crafty	6.898	5.837
197.parser	9.245	10.008
201.compress	7.841	7.831
202.jess	1.463	1.562
205.raytrace	2.945	2.756
209.db	4.262	3.909
213.javac	2.642	2.267
222.mpegaudio	2.604	2.188
227.mtrt	2.968	2.657
228.jack	3.077	3.033
252.eon	1.839	1.807
253.perlbmk	3.248	2.554
254.gap	3.814	3.926
255.vortex	1.090	1.222
256.bzip2	0.074	0.094
300.twolf	22.239	21.489
Average	**6.604**	**6.306**

With both fixed, target MPKI tracks direction MPKI closely (average 7.393 across the 19 traces), which is the expected relationship: a direction misprediction almost always implies a target miss too, plus a small amount of genuine BTB capacity miss. gcc shows the widest gap (13.302 direction vs. 18.562 target), consistent with a compiler's unusually large, varied code footprint stressing a fixed-size BTB harder than the other benchmarks.

Part 2: Tomasulo Algorithm with Multi-Issue

4 Starting Point and What Changed

The provided simulator already implements Tomasulo's algorithm correctly for single issue: register renaming, a reorder buffer, per-unit reservation stations, and a load/store queue with forwarding and disambiguation. Five stages (ISSUE, EX, MEM, WB, COMMIT) and exactly one instance of each resource (one integer adder, one FP adder, one multiplier, one load/store unit, one CDB). The required change was to issue up to four instructions per cycle, in order, without breaking anything already correct.

Issue width is now a run parameter (1–4, default 1). The core change was small: the existing single-instruction issue routine is called in a bounded loop each cycle instead of once, still strictly in order. Renaming happens synchronously per instruction, so a same-cycle read-after-write dependency within a batch resolves correctly for free. A structural hazard shows up as the program

counter failing to advance, which the batch loop treats as a stop signal. That same signal, it turns out, also correctly stops a batch at a branch. I also fixed a loop-termination bug exposed more easily by wider issue (the simulator could end a cycle early if the ROB drained right as issue stalled), and added the final register/memory dump the output format requires but the original code never printed.

Five test programs, the given benchmark plus four written to isolate the reservation-station, cross-unit, and load/store-queue structural hazards independently, confirmed final architectural state is identical across all four issue widths in every case, which is the property that actually matters: issue width should change timing, never outcome. Completion time isn't always monotonic in issue width (a single-FU bottleneck can make bursty issue mildly *slower* than steady one-per-cycle issue, confirmed directly with a reservation-station saturation test: 9 cycles at width 1 versus 11 at width 4), which is expected queueing behavior once issue bandwidth exceeds what the back end can actually drain, not a defect.

5 Branch Prediction and Misprediction Recovery

The configuration the assignment describes includes a one-bit predictor per branch, an 8-entry BTB indexed by the low 3 bits of the branch's word address, prediction at issue, resolution at the end of EX, and full misprediction recovery (RAT restore, reservation-station and ROB cleanup, refetch starting two cycles after detection). The simulator as given didn't implement any of this; it simply stalled fetch at every branch until it resolved, which sidesteps the need for recovery entirely but doesn't match the specified architecture. I implemented the full version.

5.1 Design

Each BTB entry stores a valid bit, a tag (the branch's own PC, to detect aliasing when two branches share the same low 3 bits), a one-bit prediction, and a target. Prediction happens at issue: on a tag hit, the branch follows the stored prediction and target immediately, with no stall; on a miss, it predicts not-taken, which needs no redirection since fall-through is already the default next fetch. Each branch's ROB entry also carries a snapshot of the RAT (both integer and floating-point) taken at its own issue time, for restoring on misprediction. The branch resolves in EX as before; actual outcome and target are compared against the prediction, the BTB is trained unconditionally, and on a mismatch, every ROB entry issued after the branch is identified and flushed, along with the corresponding reservation-station, load/store-queue, functional-unit, results-buffer, and CDB entries, before the RAT is restored from the branch's snapshot and fetch redirected to the correct target.

5.2 Two Real Bugs, Found by Testing

The first version deadlocked on the given benchmark program, which loops through the same branch repeatedly. Tracing cycle-by-cycle showed issue permanently stuck: four reservation-station entries sat busy forever, each waiting on an operand tag that would never broadcast. The cause was a stale RAT snapshot. A branch's snapshot is captured once, at its own issue time, as a mix of resolved values and pending tags; if a tag it holds later resolves *and commits* (its producing instruction retiring and leaving the ROB) before the branch itself resolves, the snapshot still holds that now-dead tag. Restoring from it on misprediction left a freshly issued instruction waiting on a producer that no longer existed. The fix, the same one real checkpoint-based recovery uses:

every CDB broadcast now also patches any value matching its tag into every still-pending branch's snapshot, keeping snapshots consistent with reality even for branches that haven't resolved yet.

With that fixed, a second bug surfaced only under certain issue widths: a wrong-path store's data reached the architectural register file despite being on the losing side of a misprediction. The cause was a timing race between squash and commit. My first design deferred squashing by one cycle, to match the assignment's "recovery takes one cycle, fetch resumes at n+2" rule literally. But the existing commit logic can retire several ROB entries in a single cycle once a store commits and reveals the next entry ready. Since a branch commits as soon as it resolves, regardless of whether the prediction was right, this chain could retire multiple wrong-path instructions in the very same cycle as detection, before a deferred squash ever got to intervene. The fix was to separate two things I'd conflated: cleanup cannot wait, since commit and memory writes are irrevocable, but *when fetch is allowed to resume* can. Squashing (clearing the ROB, reservation stations, load/store queue, in-flight functional units, and restoring the RAT) now happens immediately, in the same cycle the branch resolves, before that cycle's commit stage ever runs. Only `PC.valid` staying 0 through the next cycle, so fetch observably resumes at n+2, is deferred. I confirmed this with a program constructed specifically to race a wrong-path store against detection (Table 3): at every issue width, the store's data never reaches memory, and R2, written by another wrong-path instruction, correctly retains its unwritten initial value instead of the wrong-path write that leaked through before the fix.

Table 3: Wrong-path squash test: committed instructions and final R2, all widths.

	Committed	Final R2
All widths 1–4	`Addi R1`, `Bne`, `Addi R3`	32 (unwritten, correct)

5.3 Sample Output

Table 4 is the instruction status table the simulator prints for the given program at issue width 1, the output format the assignment specifies: ISSUE/EX/MEM/WB/COMMIT cycles per instruction, followed by final register and non-zero memory values. The two Bne resolutions whose EX column spans more than one cycle (`[9,9]` and `[19,19]`, versus the single-cycle `[14,14]` in between) are the two real mispredictions: the first a cold-start miss with no BTB entry yet, the third an exit miss once the loop condition changes. The final architectural state, identical at every issue width: $R1$=0, $R2$=20, $F2$=1.0, $F4$=3.0, $F6$=7.0, $F20$=2.0, all other registers 0, and the only memory value changed from its initial contents is Mem[32]=7.0 (the `Sd`), matching a by-hand check of the arithmetic.

5.4 Results by Test and Issue Width

Table 5 gives the completion cycle (last commit) for every test program at every issue width, plus how many mispredictions each one triggers. The four non-branching tests show 0 mispredictions by construction, since none of them contain a `Bne` at all, so this column is a sanity check, not a measurement. `code.in` and `test_branch_squash` are the two that actually exercise prediction and recovery.

Final register and memory state is identical across all four widths for every test in this table, confirmed by diffing the full dump rather than just eyeballing cycle counts. `test_rs_full` is the clearest illustration of *why* wider issue doesn't always mean fewer cycles. It only exercises the

Table 4: Instruction status table, given program, issue width 1.

Instruction	ISSUE	EX	MEM	WB	COMMIT
`Ld F2 0(R1)`	1	2,2	3,7	9	10
`Mult.d F4 F2 F20`	2	10,24	–	25	26
`Ld F6 0(R2)`	3	4,4	8,12	14	27
`Add.d F6 F4 F6`	4	26,29	–	30	31
`Sd F6 0(R2)`	5	6,6	32,36	–	32
`Addi R1 R1 -4`	6	7,7	–	8	33
`Addi R2 R2 -4`	7	8,8	–	10	34
`Bne R1 R0 -12`	8	9,9	–	–	–
`Addi R1 R1 -4`	11	12,12	–	13	35
`Addi R2 R2 -4`	12	13,13	–	15	36
`Bne R1 R0 -12`	13	14,14	–	–	–
`Addi R1 R1 -4`	14	15,15	–	16	37
`Addi R2 R2 -4`	15	16,16	–	17	38
`Bne R1 R0 -12`	16	19,19	–	–	–
`Add.d F20 F2 F2`	21	22,25	–	26	39

Table 5: Completion cycle by issue width, and mispredictions, all six tests.

Test	W=1	W=2	W=3	W=4	Mispredictions
`code.in` (given)	39	38	38	39	2
`test_rs_full`	9	11	11	11	0
`test_parallel_fu`	21	20	19	19	0
`test_mixed_ops`	26	24	24	23	0
`test_ldsd_full`	39	39	39	39	0
`test_branch_squash` (new)	9	9	9	9	1

4-entry integer-adder reservation station, backed by a single integer-adder functional unit, so no issue width can push more than one instruction per cycle through the actual bottleneck. Bursting four in at once (width 4) even costs one cycle more than steady single issue: the burst outruns what the single functional unit can drain, and the existing first-ready (not oldest-first) dispatch policy lets later arrivals cut ahead of earlier ones still waiting. That dispatch policy predates this assignment and is unchanged here; it doesn't affect correctness, since Tomasulo's tag-based dependency resolution only requires that no instruction consumes an operand before it's ready, which holds regardless of dispatch order. `test_mixed_ops` and `test_parallel_fu`, which spread work across the integer adder, FP adder, and FP multiplier instead of contending for one unit, show the more typical pattern of wider issue modestly helping.

Conclusion

Part 1. The required predictor combines a bimodal table and a tagged four-way global table with the Pentium M's own precedence, verified both by a hand-worked trace at the level of individual branches and by a full benchmark run against a known baseline. The extra-credit target predictor builds on it and required finding and fixing two real problems, one in my own understanding of the trace format and one in the provided infrastructure, to work correctly at all.

Part 2. Multi-issue needed a small, localized change to the issue loop. Branch prediction and misprediction recovery needed considerably more, and getting it right required finding and fixing two genuine correctness bugs, a stale recovery snapshot and a cleanup/commit race, neither of which was obvious until testing exposed them directly. Final architectural state is unaffected by issue width across every test, with or without speculation involved, which is the property that actually demonstrates correctness here.

References

[1] M. Milenkovic, A. Milenkovic, and J. Kulick, *Demystifying Intel Branch Predictors*, Workshop on Duplicating, Deconstructing, and Debunking (WDDD), 2009.